

DISPENSA DELLA LEZIONE

Richieste HTTP con JavaScript

Web 2.0, metodi HTTP, header, status code e la funzione fetch().

CAPITOLO 1

Web 2.0 e le richieste HTTP

NOTA

Con **Web 2.0** si intende la capacità delle pagine web di richiedere informazioni ai server in modo dinamico, senza dover ricaricare l'intera pagina. Questo ha reso possibile creare applicazioni web interattive e reattive come quelle a cui siamo abituati oggi.

Nel web delle origini, ogni aggiornamento di contenuto richiedeva il caricamento completo di una nuova pagina dal server. Con il Web 2.0 le cose cambiano: la pagina rimane aperta e, quando ha bisogno di nuovi dati, li richiede al server in background, ricevendo solo le informazioni necessarie e aggiornando solo la parte della pagina che cambia.

Questo tipo di comunicazione prende il nome di **richiesta HTTP asincrona**. Il termine **asincrona** significa che la richiesta viene inviata e il browser non si blocca ad aspettare la risposta: continua a funzionare normalmente, e quando la risposta arriva viene elaborata.

Il primo strumento che ha reso possibile tutto questo si chiamava **XMLHttpRequest** (spesso abbreviato in *XHR*), introdotto da Microsoft nei primi anni 2000 e poi adottato da tutti i browser. Era potente ma complesso da usare: richiedeva molte righe di codice anche per fare operazioni semplici.

Nel **2015**, con l'arrivo di ES6, è stato introdotto il **Fetch API**, un'interfaccia moderna e molto più semplice da usare. `fetch()` ha sostituito `XMLHttpRequest` nella maggior parte dei progetti, rendendo il codice più pulito e leggibile.

Esempio

La stessa richiesta scritta prima con `XMLHttpRequest` e poi con `fetch()`:

```
// Con XMLHttpRequest
const xhr = new XMLHttpRequest();
xhr.open('GET', 'https://jsonplaceholder.typicode.com/posts/1');
xhr.onload = () => {
  if (xhr.status === 200) {
    const data = JSON.parse(xhr.responseText);
    console.log(data.title);
  }
};
xhr.send();

// Con fetch()
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then(response => response.json())
  .then(data => console.log(data.title));
```

CAPITOLO 2

Osservare le richieste con i DevTools

NOTA

I **DevTools** (strumenti per sviluppatori) del browser permettono di osservare in tempo reale tutte le richieste HTTP effettuate dalla pagina. Si aprono con F12 oppure con il tasto destro del mouse sulla pagina e selezionando *Ispeziona*.

Ogni browser moderno include i DevTools, accessibili tramite F12 o con la combinazione Ctrl + Shift + I (su Windows e Linux) oppure Cmd + Option + I (su macOS). Una volta aperti, la scheda da selezionare è quella chiamata **Rete** (in italiano) o **Network** (in inglese, nella maggior parte dei browser).

In questa scheda vengono elencate tutte le richieste effettuate dalla pagina nel momento in cui viene caricata o durante l'interazione dell'utente. Per ogni richiesta è possibile vedere: l'URL, il metodo HTTP usato, il codice di stato ricevuto, la dimensione della risposta e il tempo impiegato.

Per concentrarsi sulle sole richieste HTTP delle API — escludendo immagini, CSS, font e script — si usa il filtro **Fetch/XHR** disponibile nella barra dei filtri della scheda Network. In questo modo restano visibili solo le richieste asincrone effettuate con `fetch()` o con `XMLHttpRequest`.

Cliccando su una singola richiesta si apre un pannello di dettaglio con quattro sezioni principali utili per il debug:

- **Headers** — mostra gli header della richiesta e della risposta, il metodo e il codice di stato
- **Preview** — mostra il corpo della risposta formattato (ad esempio come albero JSON)
- **Response** — mostra il corpo della risposta come testo grezzo
- **Timing** — mostra quanto tempo ha impiegato ogni fase della richiesta

CAPITOLO 3

HTTP Status Code

NOTA

Ogni richiesta HTTP riceve sempre una risposta dal server. Insieme ai dati (o in assenza di essi), il server restituisce un **codice di stato** numerico che descrive l'esito dell'operazione.

Quando il client invia una richiesta HTTP, il server elabora la richiesta e risponde sempre. La risposta contiene un **codice di stato** (status code): un numero a tre cifre che indica se la richiesta è andata a buon fine, se c'è stato un errore e di che tipo. Conoscere questi codici è essenziale per capire cosa sta succedendo tra client e server.

I codici sono raggruppati per centinaia in cinque categorie principali. Le prime due cifre indicano la categoria, la terza cifra specifica il tipo preciso di risposta all'interno di quella categoria.

Categoria	Significato generale	Codici comuni	Descrizione
1xx	Informazionale	100 Continue	Il server ha ricevuto la richiesta, elaborazione in corso
2xx	Successo	200 OK	Richiesta riuscita, i dati sono nella risposta
		201 Created	Risorsa creata con successo (tipico di POST)
		204 No Content	Richiesta riuscita, ma la risposta non contiene dati (tipico di DELETE)
3xx	Reindirizzamento	301 Moved Permanently	La risorsa si trova ora a un altro URL in modo permanente
		302 Found	La risorsa si trova temporaneamente a un altro URL
4xx	Errore del client	400 Bad Request	La richiesta è malformata o contiene dati non validi
		401 Unauthorized	Autenticazione richiesta: l'utente non è riconosciuto
		403 Forbidden	Accesso vietato: l'utente è riconosciuto ma non ha i permessi
		404 Not Found	La risorsa richiesta non esiste a quell'URL
5xx	Errore del server	500 Internal Server Error	Errore generico lato server durante l'elaborazione
		503 Service Unavailable	Il server non è disponibile, ad esempio per manutenzione

CAPITOLO 4

Metodi HTTP

NOTA

Il **metodo HTTP** indica al server che tipo di operazione il client vuole eseguire sulla risorsa indicata dall'URL. I metodi principali sono **GET**, **POST**, **PUT**, **PATCH** e **DELETE**.

Ogni richiesta HTTP è composta da un URL (l'indirizzo della risorsa) e da un metodo che descrive l'intenzione del client. Puoi immaginare l'URL come l'indirizzo di un oggetto e il metodo come l'azione che vuoi compiere su quell'oggetto: leggerlo, crearne uno nuovo, modificarlo o eliminarlo.

Metodo	Operazione	Usa il body?	Status tipico
GET	Legge una risorsa	No	200 OK
POST	Crea una nuova risorsa	Sì	201 Created
PUT	Sostituisce completamente una risorsa	Sì	200 OK
PATCH	Modifica parzialmente una risorsa	Sì	200 OK
DELETE	Elimina una risorsa	No	204 No Content

CAPITOLO 5

La funzione fetch()

NOTA

`fetch()` è la funzione JavaScript per inviare richieste HTTP. Riceve come primo argomento l'URL della risorsa e restituisce una **Promise**, cioè un valore che non è subito disponibile ma arriverà in futuro.

Una **Promise** rappresenta un'operazione asincrona: quando la risposta del server arriva, la Promise si risolve e il risultato viene passato alla funzione dentro `.then()`. I metodi `.then()` si possono concatenare: il primo riceve la risposta HTTP grezza, il secondo riceve i dati effettivi dopo averli trasformati.

Esempio

```
fetch('https://jsonplaceholder.typicode.com/posts/1')  
  .then(response => response.json())  
  .then(data => console.log(data));
```

CAPITOLO 6

Il primo `.then()`: l'oggetto `response`

NOTA

Il primo `.then()` riceve un oggetto `response` che rappresenta la risposta HTTP grezza del server. Non contiene ancora i dati veri: contiene le informazioni sulla risposta (status, header) e i metodi per leggere il corpo.

Quando la Promise di `fetch()` si risolve, passa al primo `.then()` un oggetto `Response`. Questo oggetto descrive la risposta del server prima ancora di leggerne il contenuto: è come ricevere una busta ancora chiusa, di cui però si può già vedere il mittente e il timbro postale.

Le proprietà più importanti di `response` sono:

- `response.status` — il codice di stato numerico (es. `200`, `404`)
- `response.statusText` — il messaggio testuale associato (es. `"OK"`, `"Not Found"`)
- `response.ok` — un booleano: vale `true` se lo status è tra `200` e `299`, `false` altrimenti
- `response.headers` — l'oggetto che contiene gli header della risposta, leggibili con `.get('nome-header')`

Per leggere il corpo della risposta si usano i metodi asincroni dell'oggetto `response`. Ciascuno di questi metodi restituisce a sua volta una Promise, per cui va preceduto da `return` dentro il `.then()` per passare il risultato al `.then()` successivo:

- `response.json()` — analizza il corpo come JSON e lo trasforma in un oggetto JavaScript
- `response.text()` — legge il corpo come testo semplice
- `response.blob()` — legge il corpo come dato binario (utile per immagini o file)

Il metodo da usare dipende dal formato in cui il server ha risposto. Nella maggior parte delle API moderne si usa `response.json()`.

Esempio

```
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then(response => {
    console.log(response.status);           // 200
    console.log(response.statusText);       // "OK"
    console.log(response.ok);              // true

    const tipo = response.headers.get('Content-Type');
    console.log(tipo);                     // "application/json; charset=utf-8"

    return response.json();
  })
  .then(data => console.log(data));
```

CAPITOLO 7

Il secondo `.then()`: i dati

NOTA

Il secondo `.then()` riceve il risultato del metodo chiamato nel primo `.then()`. Se è stato usato `response.json()`, riceve un oggetto JavaScript pronto da usare.

Il secondo `.then()` è il punto in cui si lavora con i dati veri della risposta. A questo punto il corpo della risposta è già stato letto e trasformato: se nel primo `.then()` è stato chiamato `response.json()`, il parametro del secondo `.then()` sarà un oggetto JavaScript con tutte le proprietà del JSON ricevuto.

Da questo momento in poi si può usare il parametro come qualsiasi altro oggetto JavaScript: leggere proprietà, stamparle in console, mostrarle nella pagina o salvarle in una variabile.

Esempio

```
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then(response => response.json())
  .then(data => {
    console.log(data.id);           // 1
    console.log(data.title);        // "sunt aut facere..."
    console.log(data.body);         // testo del post
    console.log(data.userId);       // 1
  });
```

CAPITOLO 8

Usare i metodi HTTP con fetch()

NOTA

Con `fetch()` il metodo predefinito è `GET`. Per usare un metodo diverso si passa un secondo argomento alla funzione: un oggetto di configurazione con la proprietà `method` e, se necessario, `body`.

L'oggetto di configurazione può contenere tre proprietà principali: `method` per il metodo HTTP, `body` per i dati da inviare al server (obbligatorio per POST, PUT e PATCH), e `headers` per gli header della richiesta. Il body deve essere una stringa: per inviare un oggetto JavaScript lo si converte con `JSON.stringify()`.

Esempio

```
// POST - crea una nuova risorsa
fetch('https://jsonplaceholder.typicode.com/posts', {
  method: 'POST',
  body: JSON.stringify({ title: 'Nuovo post', userId: 1 }),
  headers: { 'Content-Type': 'application/json' }
})

.then(response => response.json())
.then(data => console.log(data));

// PATCH - modifica solo il titolo
fetch('https://jsonplaceholder.typicode.com/posts/1', {
  method: 'PATCH',
  body: JSON.stringify({ title: 'Titolo aggiornato' }),
  headers: { 'Content-Type': 'application/json' }
})

.then(response => response.json())
.then(data => console.log(data));

// DELETE - elimina la risorsa
fetch('https://jsonplaceholder.typicode.com/posts/1', {
  method: 'DELETE'
})

.then(response => console.log('Status: ' + response.status));
```

CAPITOLO 9

Header HTTP

NOTA

Gli **header** sono informazioni aggiuntive che viaggiano insieme alla richiesta o alla risposta HTTP. Non fanno parte del corpo del messaggio, ma descrivono come interpretarlo.

Puoi immaginare la richiesta HTTP come una busta: il **body** è la lettera dentro la busta, mentre gli **header** sono le informazioni scritte sulla busta stessa (mittente, tipo di contenuto, lingua, ecc.). Il server legge gli header prima di aprire la busta, per sapere come gestire il contenuto.

Gli header sono coppie chiave-valore. Con `fetch()` si impostano tramite la proprietà `headers` nell'oggetto di configurazione. L'header più comune nelle richieste è `Content-Type`, che indica al server in quale formato sono i dati inviati nel body. Quando si inviano dati JSON, il valore è `application/json`. Gli header della risposta si leggono con `response.headers.get('nome-header')`.

Header	Direzione	Scopo	Esempio di valore
<code>Content-Type</code>	Richiesta e risposta	Specifica il formato dei dati nel body	<code>application/json</code>
<code>Authorization</code>	Richiesta	Trasmette credenziali di autenticazione	<code>Bearer il-mio-token</code>
<code>Accept</code>	Richiesta	Indica al server il formato di risposta atteso	<code>application/json</code>
<code>Cache-Control</code>	Richiesta e risposta	Controlla il comportamento della cache	<code>no-cache</code>

Esempio

```
fetch('https://jsonplaceholder.typicode.com/posts', {
  method: 'POST',
  body: JSON.stringify({ title: 'Test', userId: 1 }),
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer il-mio-token'
  }
})
.then(response => {
  const contentType = response.headers.get('Content-Type');
  console.log('Tipo di risposta: ' + contentType);
  return response.json();
})
.then(data => console.log(data));
```